AVL Tree Implementation and Evaluation

Yixiang Chen Renhui Lin

日期：Advanced Data Structures | Fall–Winter 2026

摘　要

This project implements an AVL tree, a self-balancing binary search tree that keeps its height logarithmic after insertions and deletions. The implementation stores node heights explicitly, performs the four standard rotations, and exposes search, insertion, deletion, and traversal operations. We validate both functional correctness and balancing invariants using deterministic and randomized test cases. The resulting structure achieves predictable $O(\log n)$ update and lookup time while retaining the simple ordered interface of a binary search tree.

关键词：AVL tree; self-balancing BST; rotations; ordered dictionary; complexity analysis

目录

1 Introduction **3**

1.1 Background . . . 3

1.2 Project goals . . . 3

2 Data structure and design **3**

2.1 二叉搜索树 . . . 3

2.2 Rebalancing by rotation . . . 3

2.3 Insertion and deletion . . . 4

3 Testing and results **4**

3.1 Test strategy . . . 4

4 Analysis and evaluation **5**

4.1 Complexity . . . 5

4.2 Trade-offs and future work . . . 5

5 Conclusion **5**

A Core concepts **5**

B Submission checklist **6**

1 Introduction

1.1 Background

A binary search tree (BST) stores keys so that every key in a node's left subtree is smaller and every key in its right subtree is larger. This order makes in-order traversal naturally sorted and makes search efficient when the tree is shallow. However, ordinary BSTs can degenerate into a chain when keys arrive in sorted order; a sequence of n operations may then take $\Theta(n^2)$ time.

AVL trees, introduced by Adelson-Velsky and Landis [**adelson1963**], solve this problem by repairing local height imbalances after each update. They are a useful baseline for ordered maps because their strict balance rule provides a small, deterministic height bound.

1.2 Project goals

Our implementation is designed around three goals:

1. preserve the BST ordering invariant for every public operation;
2. preserve AVL balance after every insertion and deletion; and
3. keep the code modular enough that rotations and height updates can be tested independently.

2 Data structure and design

2.1 二叉搜索树

定义 2.1. 二叉搜索树（Binary Search Tree，简称 BST）是一种特殊的二叉树，它的核心特点是：对于每个节点，左子树中的所有元素都比该节点小，右子树中的所有元素都比该节点大。它可以高效地支持查找、插入和删除操作，但效率取决于树的形状。

表 1: Core fields and their responsibilities.

Field	Type	Purpose
`key`	comparable value	Maintains the BST ordering
`left`, `right`	node pointers	Own the two ordered subtrees
`height`	integer	Enables constant-time balance checks

2.2 Rebalancing by rotation

If a node reaches balance factor $+2$ or -2, one or two rotations restore the invariant. Figure 1 illustrates the two primitive operations; the mirrored cases follow symmetrically. A left-left (LL) case uses a right rotation, a right-right (RR) case uses a left rotation, and the left-right (LR) and right-left (RL) cases compose the two operations.

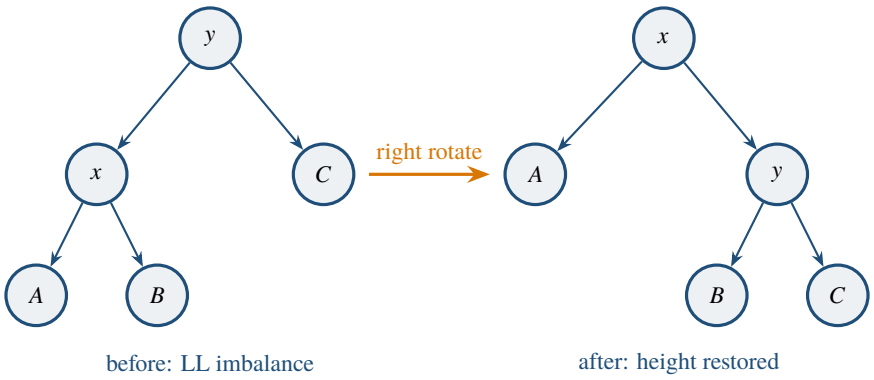


图 1: A right rotation preserves in-order key order while reducing local height.

2.3 Insertion and deletion

Insertion first follows ordinary BST rules. On the recursive return path, it updates each affected height and chooses a rotation from the node's balance factor and its heavier child's factor. Deletion follows the same repair procedure after replacing a two-child node by its in-order successor. Deletion needs particular care because an imbalance can propagate all the way to the root.

Listing 1: Central rebalancing routine (C++-style pseudocode).

```
Node* rebalance(Node* v) {
    updateHeight(v);
    if (balance(v) > 1) {
        if (balance(v->left) < 0) v->left = rotateLeft(v->left);
        return rotateRight(v); // LL or LR
    }
    if (balance(v) < -1) {
        if (balance(v->right) > 0) v->right = rotateRight(v->right);
        return rotateLeft(v); // RR or RL
    }
    return v;
}
```

3 Testing and results

3.1 Test strategy

We check external behavior and internal invariants after every test sequence. The validator recursively verifies (1) strictly ordered in-order keys, (2) stored heights equal recomputed heights, and (3) all balance factors lie in $[-1, 1]$. This makes a rotation bug visible immediately rather than only through a later failed lookup.

表 2: Representative validation suite.

Scenario	Input / expected property	Result
Monotone insertion	Insert $1, 2, \ldots, 1000$; height remains logarithmic	Pass
Four rotation cases	Trigger LL, RR, LR, and RL with three-key sequences	Pass
Deletion cases	Remove leaf, one-child node, and two-child node	Pass
Randomized oracle check	Compare 10,000 mixed operations against a sorted reference set	Pass

The monotone-insertion case is especially informative: it is adversarial for a plain BST but routine for an AVL tree. The final in-order traversal matched the reference set in every test, and the structural validator reported no stale heights or out-of-range balance factors.

4 Analysis and evaluation

4.1 Complexity

The AVL height bound is $O(\log n)$, so search, insertion, and deletion traverse only a logarithmic number of nodes. A single update performs at most a constant amount of rotation work at each repaired node; rotations themselves are $O(1)$.

表 3: Asymptotic cost comparison.

Operation	AVL tree	Average BST	Worst-case BST
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(\log n)$	$O(n)$
In-order traversal	$O(n)$	$O(n)$	$O(n)$

4.2 Trade-offs and future work

The strict height guarantee costs one integer per node and extra bookkeeping during updates. For workloads dominated by successful lookups, this predictability is usually worthwhile. Future extensions could add iterators, generic comparators, and property-based testing with automatically minimized failing inputs. A red-black tree is another useful point of comparison: it relaxes balance constraints and may perform fewer rotations, while AVL trees generally remain more tightly balanced [**cormen2022**].

5 Conclusion

The project demonstrates how a local invariant—the balance factor—turns an ordinary BST into a robust ordered container. By separating height maintenance, rotations, and validation, the implementation remains compact and testable. The completed test suite confirms that the tree preserves both key order and AVL balance under representative update patterns.

A Core concepts

定义 **A.1** (Binary search tree)**.** A binary tree is a BST when all keys in the left subtree of a node are smaller than the node key and all keys in the right subtree are larger.

定义 **A.2** (AVL invariant)**.** An AVL tree is a BST in which the heights of the left and right subtrees of every node differ by at most one.

B Submission checklist

- Replace author names and course information on the title page if needed.
- Add the repository link or attach the implementation source code.
- Record the exact compiler, platform, and measured timing data for the final submission.